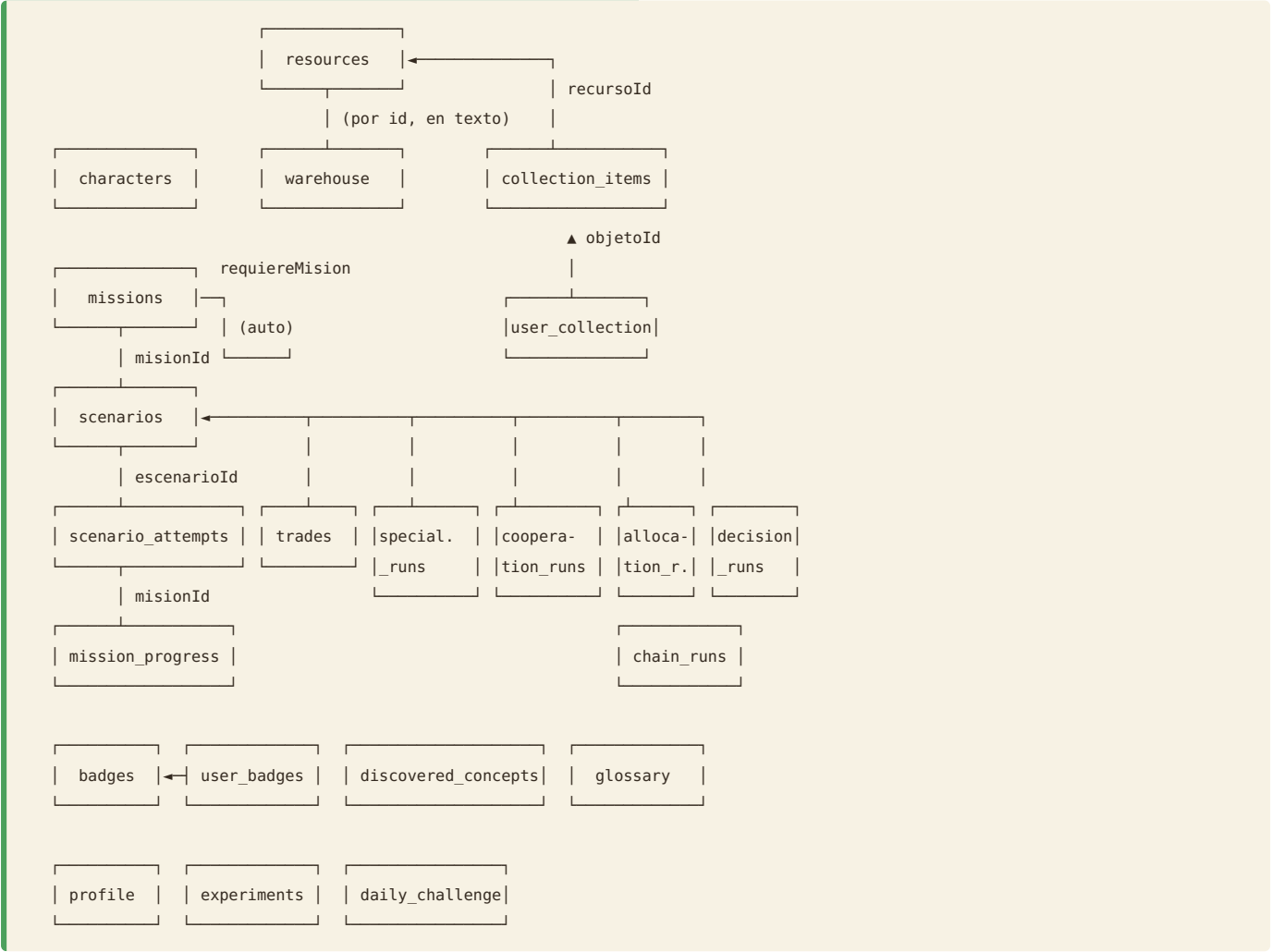


Base de datos — RedEconómica v1.0.0

Motor: **SQLite** a través de **Room 2.6.1**. Archivo en el dispositivo: `redeconomica.db`. Versión de esquema: **1**. 22 tablas. Todo local: no hay servidor, ni sincronización, ni copia gestionada por la aplicación.

El SQL equivalente está en `database/schema.sql` y el contenido sembrado en `database/sample_data.sql` (generado desde el código por `tools/generar_sample_data.py`).

1. Diagrama



Room no declara claves foráneas: las relaciones se mantienen por id desde los repositorios y se verifican en pruebas (`SeedContent.problemasDeIntegridad()`). Se prefirió así para que el sembrado sea idempotente y el borrado de progreso no arrastre el catálogo.

2. Tablas de catálogo

Se siembran una sola vez, en el primer arranque, desde `data/seed` mediante `DatabaseSeeder`. Volver a arrancar la app no duplica nada.

Tabla	Filas sembradas	Qué guarda
resources	20	Recursos del Valle: id, singular, plural, familia, <code>valorBase</code> , descripción
characters	9	8 vecinos + Tilo: oficio, zona, productividad, inventario base, avatar
missions	14	Las misiones: número, zona, concepto, narrativa, requisito, recompensa
scenarios	42	Ficha de cada desafío: misión, orden, tipo, textos, concepto, dificultad
badges	11	Insignias: regla que las desbloquea y meta
collection_items	24	Objetos del Almacén del Valle
glossary	13	Diccionario del Valle

Nota de diseño: dónde viven las reglas de un escenario

En `scenarios` está la **ficha** de cada desafío, y de ella salen los listados, el orden dentro de la misión y el progreso. Las **reglas** concretas —qué tiene cada personaje, cuántas etapas hay, qué opciones de presupuesto existen— viven en el módulo de contenido `data/seed`, **fuera de la interfaz**, porque tienen forma distinta según la mecánica (una escasez y una cadena de producción no comparten columnas). Guardarlas en tablas habría exigido escribir un serializador propio sin ganar ninguna consulta útil.

Es una simplificación consciente y está declarada también en la memoria descriptiva. Lo que **no** se hizo es meter esas reglas dentro de los Composables: son datos, se validan con `ScenarioValidator` y las pruebas los recorren enteros.

3. Tablas de estado

Tabla	Clave	Qué guarda
profile	<code>id = 1</code>	Alias, avatar, onboarding, ajustes. Ningún dato personal.
warehouse	<code>recursoId</code>	Recursos acumulados por el jugador
mission_progress	<code>misiónId</code>	Estado de la misión, intentos, si fue sin fallos
scenario_attempts	auto	Un registro por intento: escenario, misión, concepto, si se logró
trades	auto	Cada propuesta de intercambio, aceptada o no, con su motivo
specialization_runs	auto	Cada reparto de trabajo ejecutado, con su producción y valor
cooperation_runs	auto	Cada trabajo en equipo, con resultado, objetivo y lo que saldría sin cooperar
allocation_runs	auto	Cada reparto de un recurso escaso
decision_runs	auto	Cada decisión con presupuesto, con sus renuncias
chain_runs	auto	Cada intento de ordenar una cadena
user_badges	<code>insigniaId</code>	Insignias conseguidas
user_collection	<code>objetoId</code>	Objetos desbloqueados
discovered_concepts	<code>conceptoId</code>	Conceptos ya vividos
experiments	auto	Cada experimento del Laboratorio, con su configuración y resultado
daily_challenge	<code>diaIndice</code>	Reto del día y si se completó

4. Formatos compactos

Tres tipos de dato se guardan como texto legible en lugar de crear tablas auxiliares:

Formato	Ejemplo	Dónde
Inventario / productividad	<code>manzana:6 verdura:3</code>	<code>characters.productividad</code> , <code>trades.entrega</code> , <code>warehouse</code> (no), <code>experiments.produccionTotal</code>
Plan de trabajo	<code>lia=manzana tomas=pan nina=</code> (vacío = «hace de todo»)	<code>specialization_runs.plan</code> , <code>cooperation_runs.plan</code>
Lista simple	<code>silla cesta</code>	<code>decision_runs.seleccion</code> , <code>chain_runs.ordenPropuesto</code>

Se eligió este formato porque es **legible desde cualquier visor de SQLite**, va y vuelve sin pérdida (probado en `InventoryTest`) y la aplicación nunca necesita consultar «dame todos los intercambios que incluyan manzanas»: siempre lee

la fila entera.

Los enumerados se guardan con el nombre de la constante Kotlin (`ALIMENTO` , `COMPLETADO` , `DESEQUILIBRIO`), no con números, para que el volcado se entienda sin abrir el código.

5. De dónde sale cada número del perfil

Ningún contador se guarda. Todo se calcula contando filas:

Lo que ve el niño	Consulta
Intercambios aceptados	<code>SELECT COUNT(*) FROM trades WHERE aceptado = 1</code>
Intercambios propuestos	<code>SELECT COUNT(*) FROM trades</code>
Repartos de trabajo probados	<code>SELECT COUNT(*) FROM specialization_runs</code>
Trabajos en equipo terminados	<code>SELECT COUNT(*) FROM cooperation_runs WHERE completado = 1</code>
Repartos de lo escaso resueltos	<code>SELECT COUNT(*) FROM allocation_runs WHERE valido = 1</code>
Decisiones tomadas	<code>SELECT COUNT(*) FROM decision_runs WHERE alcanza = 1</code>
Cadenas ordenadas	<code>SELECT COUNT(*) FROM chain_runs WHERE correcto = 1</code>
Experimentos	<code>SELECT COUNT(*) FROM experiments</code>
Misiones completadas	<code>SELECT COUNT(*) FROM mission_progress WHERE estado IN ('COMPLETADO', 'DOMINADO')</code>
Conceptos descubiertos	<code>SELECT COUNT(*) FROM discovered_concepts</code>
Objetos del Almacén	<code>SELECT COUNT(*) FROM user_collection</code>

Los **sellos del Valle** y el nivel se derivan de esos contadores en `ProgressEngine`, no de una columna. Las insignias se recalculan contra los contadores cada vez que ocurre algo relevante, así que no pueden quedar desincronizadas.

Los desafíos pendientes de repaso también son una consulta:

```
SELECT DISTINCT escenarioId FROM scenario_attempts
WHERE logrado = 0
AND escenarioId NOT IN (SELECT escenarioId FROM scenario_attempts WHERE logrado = 1);
```

6. Migraciones y borrado

`AppDatabase` está configurada con `fallbackToDestructiveMigration()`. Al no haber datos personales y ser el catálogo reproducible desde el código, un cambio de esquema vuelve a sembrar en lugar de arrastrar migraciones. Si en el futuro se quisiera conservar el progreso entre versiones, habría que escribir migraciones reales y subir `version`.

Desde Ajustes, «Empezar el Valle de cero» ejecuta `DatabaseSeeder.reiniciarProgreso()`, que vacía **solo** las tablas de estado y deja intacto el catálogo.

7. Comprobado de verdad

`database/schema.sql` y `database/sample_data.sql` se cargaron en un SQLite real durante la preparación de la entrega y las consultas del apartado 5 se ejecutaron con éxito sobre ellos. Recuentos verificados: 20 recursos, 9 personajes, 14 misiones, 42 escenarios, 11 insignias, 24 objetos y 13 entradas de diccionario.